

技术分享

OpenClaw vs Hermes

两大主流开源智能体（Agent）的原理、架构与实践对比

聚焦 Hermes 的「自进化」机制 —— 一个会记住你、会和你一起成长的 Agent

第一性原理

自进化循环

持久记忆

架构解析

三者横向对比

落地避坑

分享人 王宇 · 中交信科集团星宇公司

时长 60 分钟 · 第一性原理 → 系统拆解 → 横向对比 → 落地避坑 · 含 Claude Code 横向参照

中交信科集团星宇公司 · 王宇

60 分钟怎么走

01



第一性原理

智能体到底是什么 · 那个核心循环 · 5 个必备原语

10 min

02



系统拆解：Hermes 与 OpenClaw

用「5 原语」视角逐一深入：杀手铜 + 软肋

30 min

03



横向对比 + 设计抉择

三者一张大表 · 建 agent 必答的 6 个问题 · 怎么选

10 min

04



落地 · 避坑 · 经验

上手路径 · 三大坑 · 带走的三句话

10 min

PART 01

FIRST PRINCIPLES

第一性原理

智能体到底是什么 · 那个核心循环 · 5 个必备原语

10 min

心智模型

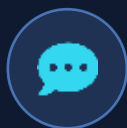
核心循环

5 原语

反直觉洞察

先分清：Chatbot / Workflow / Agent

同一句话「帮我把这个项目整理好，并修掉那个登录 bug」，三种东西的反应完全不同：



Chatbot

单轮问答

「你可以打开 auth.js，检查第 42 行的 token 校验……」

→ 给你建议，活儿你自己干。



Workflow

写死的固定步骤

lint → test → 按预设规则替换 → 提交。遇到没预设的情况就停。

→ 可靠但不灵活，碰到意外就卡住。



Agent

自己决策的闭环

读文件 → 推断 bug → 改 → 跑测试 → 看到失败 → 再改 → 通过 → 提交。

→ 自己看结果、自己决定下一步。

关键差异：agent 会「看到结果并据此改变下一步」—— 这就是「闭环」。一次任务可以自己转几十上百轮。

核心循环：所有智能体的共同底座

① 输入 / 指令

② 组装上下文

③ 调用模型

④ 执行工具

⑤ 结果喂回

🔄 重复直到收尾

模型给出「纯文本、无工具调用」时，循环自然停止。



干货：工具失败别写一堆脆弱的重试树。shell 非零退出码 + stderr 直接当结果喂回去，模型像人一样读报错、换个方法再试 —— 让有能力的模型自己闭环。



```
# 一个能打的 agent, 核心就这么点
state = [ system_prompt, tools, user_msg ]

while True:
    reply = model(state)          # 模型决定下一步
    state.append(reply)
    if not reply.tool_calls:      # 没有工具调用
        break                    # → 收尾, 交还用户
    for call in reply.tool_calls:
        result = run(call)       # 真实世界副作用
        state.append(result)     # 喂回, 进入下一轮
```

5 个必备原语：后面的系统都按这拆



① 循环 · 决策引擎

调模型 → 调工具 → 喂回，直到收尾。这一层，三个系统完全一样 —— 差异全在另外四个原语上。



② 上下文 & 记忆

它「知道」什么

塞什么进有限的窗口；跨会话怎么记住。



③ 工具

它能「做」什么

读写文件、跑命令、搜网、发消息 —— 真实副作用。



④ 权限 & 安全

什么不许做

哪些先问人、哪些直接拦、隔离与沙箱。



⑤ 持久化

什么能留下

会话日志、可恢复、可审计、可回滚。

反直觉洞察：少搭框架，多打磨外壳

Claude Code 源码拆解

1.6%

是 AI 决策逻辑（那个 while 循环 + 提示）

98.4%

是确定性「外壳」工程 —— 权限闸门 · 上下文管理 · 工具路由 · 恢复逻辑

框架不是越重越好



Anthropic 自己的建议：用简单、可组合的模式，而不是笨重框架。能力来自模型 + 外壳，不来自抽象层数。

模型在收敛，外壳在分化



各家大模型能力越来越接近。决定 agent 好不好用的，是外面那圈工程，不是又换了个底座模型。

外壳 = 真正的护城河



循环一晚上能抄完；但权限分级、上下文压缩、子 agent 隔离、失败自愈 —— 这些「横切」机制最难复制。

PART 02

SYSTEMS TEARDOWN

系统拆解：Hermes 与 OpenClaw

用「5 原语」视角逐一深入，看清各自的杀手锏与软肋

30 min

自进化循环

持久记忆

四层架构

控制总线



现象级崛起：不到两个月，冲上 10 万星



< 2 个月

从发布到刷屏的时间



10 万+

GitHub Stars



腾讯云 / 阿里云

相继推出一键部署方案



背景：OpenClaw 之后，最受瞩目的开源 Agent

OpenClaw（圈内俗称「龙虾」）火了约两个月，被认为是近两年最强的开源 Agent。紧接着 Hermes Agent 出现，不到两个月就在 GitHub 狂揽 10 万星，腾讯云、阿里云相继推出一键部署。

于是大家开始问：「Hermes 到底比龙虾强在哪？」—— 但这个问题，方向本身就错了。

它的厉害，不在「底层模型」

✗ 不是因为用了更强的模型

- ✗ 宣传里没有一条说「我们用了 GPT-6」
- ✗ 也没有「自研 1145B 大模型」
- ✗ 底层模型，恰恰不是它的卖点

传统 AI：关掉窗口就忘了。下次再说「帮我安排日程」，它不会记得你上周交代过的工作节奏与偏好。

🌱 真正的核心卖点

“The agent that grows with you”

—— 会和你一起成长的 Agent（养成系）

👤 它不是工具，更像一个学生 / 实习生

🧠 你完成一件事，它就记住你「怎么做的」

🔄 下次同类任务，直接按你的方式来，不用重复交代



核心差异：不是模型，而是「学习的方式」



传统 AI

无状态 · 用完即忘

- ✗ 关掉窗口，上下文就清空
- ✗ 每次都要从头重新交代
- ✗ 记不住你的偏好与工作习惯
- ✗ 同样的任务，每次都重跑一遍

VS



Hermes

有记忆 · 越用越懂你

- ✓ 跨会话记住关键信息
- ✓ 记住你「怎么完成」一件事
- ✓ 建立你的偏好画像与工作风格
- ✓ 下次同类任务，直接按你的方式执行



一句话：竞争点已经从「谁的模型更强」转向「谁更懂你、和你的关系更持久」。

Self-Improving Loop: 三层递进的自进化循环

这是 Hermes 最核心、也是它区别于所有其他方案的地方 —— 让 Agent 自己变聪明



三层闭环回流：进化后的能力反哺记忆与技能，让下一轮做得更快、更准 —— 这正是「grows with you」。

第一层 · 记忆积累：用户不用每次都重新解释



举个例子：把一条偏好「教」给它一次

在一个「用 AI 生成 HTML」的自动化技能里，我特别强调过一点：

× **不要** 加入土味 emoji； ✓ **应当** 改用扁平 UI 图标库的图标代替。

在 Hermes 里，只要对话几个回合，它就能彻底记住这条偏好，并自动沉淀成一个技能 (SKILL.md) —— 引出第二层。

持久记忆系统：短期 → 压缩 → 长期

跨会话召回 Cross-Session Recall



底层 SQLite FTS5 支持全文检索（可跨会话查到「上次用什么工具部署的 Docker」）；上下文有限，于是用 **LLM Summarization** 自动压缩历史、提炼要点，用更少 token 表达同样信息量；**Honcho 渐进式用户画像** 则在多次交互中慢慢拼出「你是谁、关心什么、喜欢怎样的工作方式」。

持久记忆的价值: Don't need to say it again



传统 AI

「帮我安排一场乐队的 Live 演出流程」

像临时演出策划一样，下单就完事。下次再说「安排演出」，它完全不记得乐队有什么偏好。



Hermes 记住的，是一整套偏好



Live 固定在周六下午两点开趴



开场必须配专属的 call-and-response 环节



安可曲目固定搭配（迷之曲单）

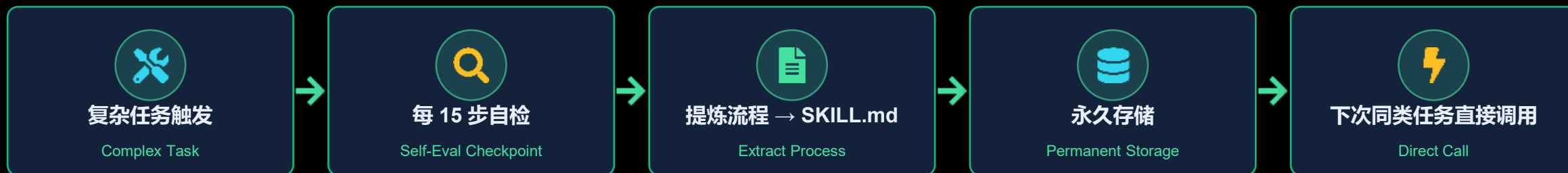


结束后带大家去常去的家庭餐厅，提前留好座位

以后再让它安排 Live，直接按你的偏好来 —— Don't need to say it again.

机制支撑：Honcho dialectic user modeling —— 不是一次性记住，而是在多轮交互中渐进式建立用户画像。

第二层 · 技能生成：把一次任务沉淀为技能



省时间

下次同类任务不必重新规划、从头来一遍。



省 Token

调用 SKILL 而不是再跑一遍大模型请求。



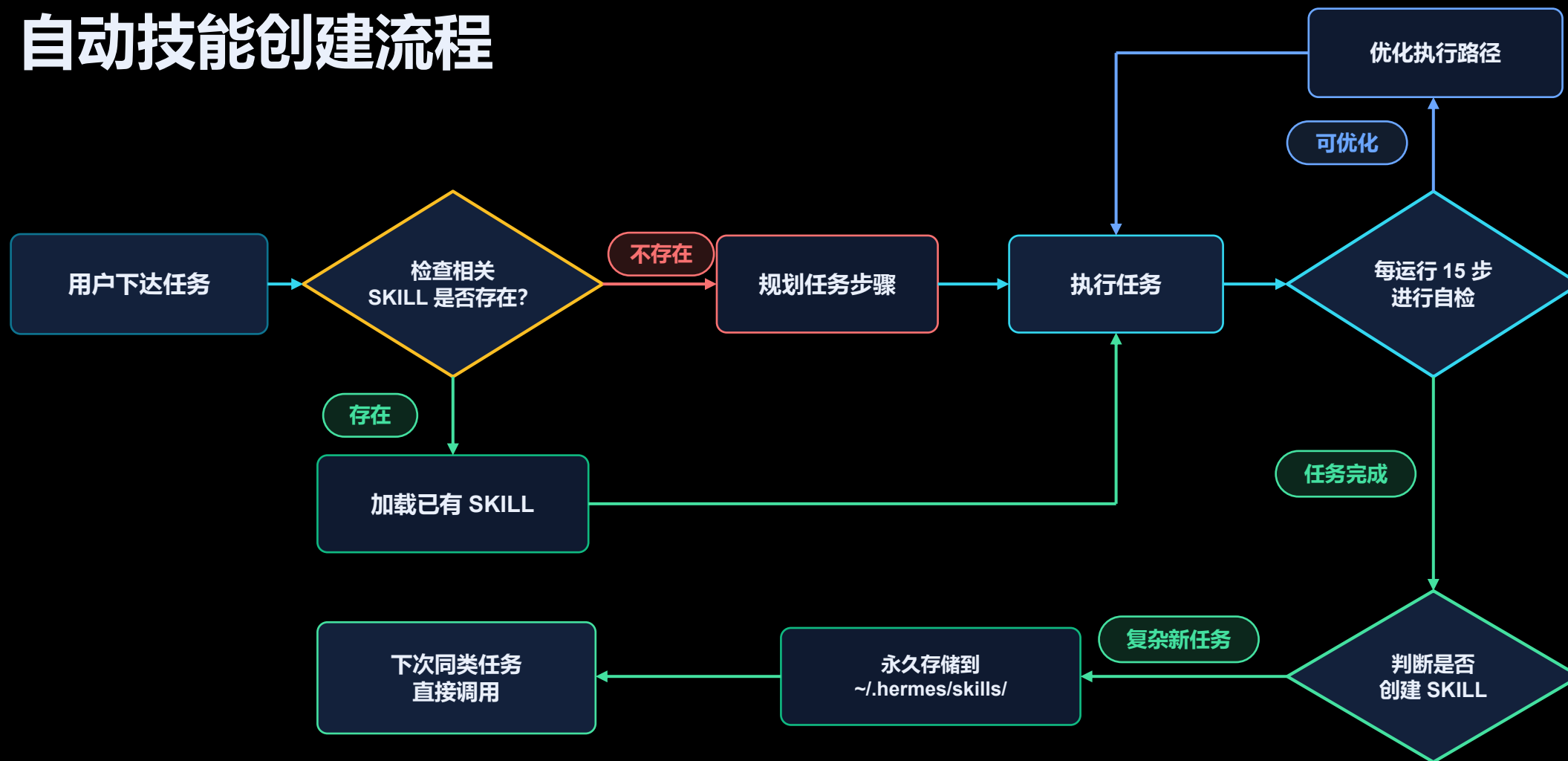
可复用

workflow 被固化成文档，反复调用稳定输出。



技能文档名为 **SKILL.md**：完成复杂任务后，Hermes 自动把「这次怎么做的、关键步骤、下次怎么做」提炼成一份永久存储的文档。

自动技能创建流程



兼容 agent-skill.io 开放标准：你写的技能能被别人用，也能复用别人分享的技能。

从「用完就忘」到「越用越强」——但有隐患



越用越强的机制

- ✓ 与 OpenClaw 不同：技能不需要你手动安装 / 配置
- ✓ 复杂任务（如「从 0 到 1 搭个人网站」）完成即自动建技能
- ✓ 技能永久存储；下次先查有无相关技能，有则直接调用
- ✓ 兼容 agent-skill.io 开放标准，可分享、可复用他人技能



一个真实的隐患

「我一个室友用了一晚上之后的反馈」



用得越久，自动生成的 skill 越积越多，反而可能导致效率下降。

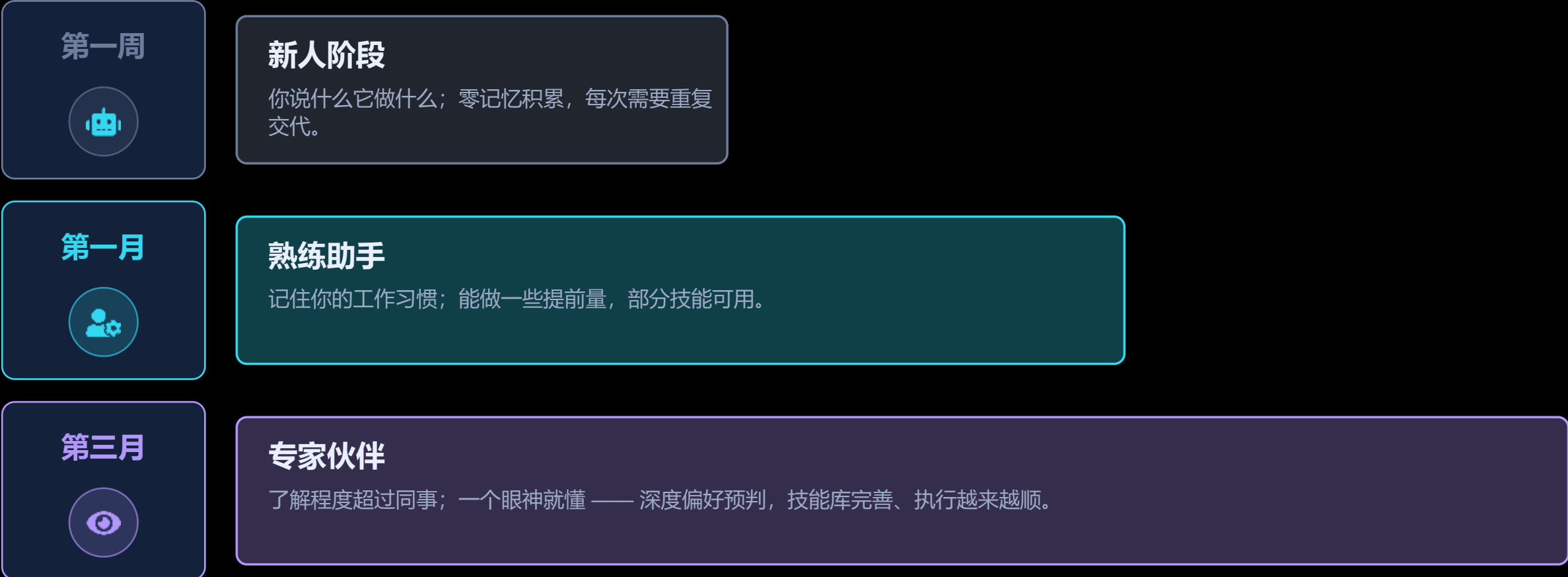
工具不是越多越好，精简才能保持高效。

这正好引出第三层 —— 持续进化：把前两层闭环跑通、自动优化。

第三层 · 持续进化：自我优化与智能预判



用户进化曲线 User Evolution Curve



这条进化曲线，正是 Hermes 最有价值的地方 —— 用得越久，它对你的了解越深。

Hermes Agent 四层架构

「每 15 次工具调用，命中一次自我进化 checkpoint，开始创建技能」—— 不被任何单一模型厂商绑定。



OpenClaw（龙虾）：被反复验证的老牌巨头



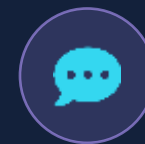
36.2 万+

GitHub Stars · 多年社区沉淀



数千+ 插件

成熟的官方 / 社区插件市场



25 渠道

全渠道统一接入能力



定位：确定性 · 可控性 · 可治理性的事实标准

OpenClaw 不靠「养成」取胜。它把「人始终在回路里」做到极致——可控、可预测、可审计、可治理，是整个生态长期围绕构建的基础设施。

如果说 Hermes 讲的是「会成长」，OpenClaw 讲的就是「我说了算」——两条都被反复验证过的路线。

核心理念：控制优先，人在决策链中心



它的底气：确定性

- ✓ 你定义流程 / 工具 / 权限边界
- ✓ Agent 严格按你给定的方式执行
- ✓ 不会「自作主张」地改变行为
- ✓ 每一步都可预期、可复盘

新人第一天和第一百天，行为一致、可控——没有「悄悄漂移」带来的惊喜。



为什么有人坚定选它

“你告诉它做什么”

—— 与 Hermes「它学会你想做什么」正好相反



合规 / 审计场景：行为必须可解释、可追溯



重控制场景：金融、内部系统、敏感数据



人是决策者，Agent 是听话的执行者

OpenClaw 架构：Gateway 控制总线

你定义 → Gateway 调度 → Agent 执行 · 每条消息都过权限校验



记忆机制：显式、可读、可版本化

人能读写的记忆文件



MEMORY.md

项目 / 任务记忆 · 上下文、约定、进度



USER.md

用户画像 · 偏好、风格、习惯

想记什么 → 写进文件；不想要 → 删掉。完全由你掌控，没有黑盒。



用透明，换掉黑盒

- ✓ 可 git 版本化 —— 每次改动可追溯 / 回滚
- ✓ 可审计 —— 团队能直接 review 记忆内容
- ✓ 无静默画像 —— 不会偷偷给你建模
- ✓ 可移植 —— 换机器 / 换人直接带走

对照：Hermes 全自动压缩 / 建画像（省心但黑盒）；OpenClaw 手动 Markdown（多一步，但透明可控）。

技能体系：成熟插件市场 + 全渠道覆盖



数千现成插件

官方 + 社区长期沉淀，覆盖 25 渠道



手动但精确

你清楚装了什么、能做什么，无意外行为



边界清晰

能力可控、可复用，适合团队协作沉淀



对照：Hermes 自动生成 SKILL.md（省事，但技能可能越积越臃肿）；OpenClaw 手动装配（上手成本更高，但能力边界清晰、行为可控）。

安全与治理：面向企业的可控型设计



DM 配对

绑定可信身份后才能对话，杜绝陌生人调用



显式 Allowlist

工具 / 命令白名单，超出范围一律拒绝



逐条消息校验

每一条指令都过一次权限校验



显式人工审批

高风险动作必须人点头才执行

理念：最小权限 + 人始终在回路。 对照 Hermes 靠「默认沙盒隔离 + 容器执行」（智能自主型）；OpenClaw 靠「流程管控 + 显式审批」（企业可控型）。

部署与适用：本地优先，掌控感拉满



部署：本地 / 自托管为主

- ✓ 主要在本地 / 自有服务器运行
- ✓ 数据与控制权都在自己手里
- ✓ Serverless 可行，但需自行搭建
- ✓ 无强制遥测，环境完全自定义



它最适合谁



企业 / 团队：需要合规、审计、权限治理



重控制场景：金融、内部系统、敏感数据



偏好「我定义、它执行」确定性的人

一句话：当你要的是「确定性与可控性」胜过「省心与自动进化」时 —— 选 OpenClaw。

PART 03

SIDE BY SIDE · DESIGN

横向对比 + 设计抉择

一张大表 · 建 agent 必答的 6 个问题 · 怎么选

10 min

两两深对比

三者一张表

6 个问题

对号入座

两种设计理念：没有谁更强，只是适合的人群不同



OpenClaw 核心理念

控制优先 · 人在决策链中心

- 人在中心：用户处于决策链中心位置
- 你定义了，Agent 才能执行
- Gateway 作为 WebSocket 管理中枢
- 统一管理 Session / 工具 / 事件
- 需要你对系统有完整配置与掌控



Hermes 核心理念

进化优先 · 从交互中自主学习

- 不用告诉它每一步怎么做
- 它从你的交互中自己学习
- 你让它做的越多，它越懂你
- 确保下一次比上一次做得更好
- 无需人工维护记忆与技能

OpenClaw = 你告诉它做什么； Hermes = 它学会你想做什么。 这是「你想让 AI 以什么方式陪伴你」的问题。

五个维度，一张表看懂

维度	OpenClaw (龙虾)	Hermes
记忆机制	纯 Markdown 文件，手动编辑	全自动压缩 / 提炼 / 建画像
技能获取	插件市场手动安装、配置、调试	任务完成后自动生成技能文档
部署成本	主要本地运行，Serverless 需自搞	Docker/SSH/Modal/Serverless, \$5 VPS 可跑
安全模型	DM 配对 + 显式 Allowlist, 逐条审批	默认沙盒隔离 + 容器执行 + 零遥测
生态规模	GitHub 36.2 万星，插件超数千、覆盖 25 渠道	GitHub 10 万星，自动技能 + 开放标准

下面逐一展开记忆·技能·部署·安全四个维度。

维度 1 · 记忆机制：手动编辑 vs 全自动管理

OPENCLAW · MANUAL

 **纯 Markdown 存储**
MEMORY.md · USER.md, 手动编辑文件

① 想要 Agent 记住内容?

② 打开 MEMORY.md

③ 手动编辑文件内容

需要人工维护

HERMES · AUTOMATIC

 **全自动记忆管理**
无需人工干预, 智能自动处理

✓ ① 与 Hermes 正常交互

✓ ② 自动压缩信息

✓ ③ 自动提炼要点

✓ ④ 自动建立用户画像

用户无需操心, Hermes 自己就记住了

维度 2 · 技能获取：手动安装 vs 自动生成

OPENCLAW · MANUAL

 **手动安装配置**
用户自行操作，需要人工处理

① 去插件市场搜索（超数千技能）

② 安装所需技能

③ 配置并调试

搜索

浏览

安装

配置

调试

HERMES · AUTO

 **技能自动生成**
无需用户动手，智能自动处理

✓ ① 让 Hermes 完成任务

✓ ② 自动提炼经验

✓ ③ 生成技能文档

 skill_自动生成.md

维度 3 · 部署成本：多种方式 vs 主要本地

HERMES · LOW COST

 **多种部署方式**
灵活选择 · 低成本方案

✓ Docker 容器部署

✓ SSH 远程连接

✓ Modal 云服务

✓ Serverless 无服务器

\$ 官方称：5 美元的 VPS 即可运行

OPENCLAW · LOCAL

 **主要是本地运行**
本地优先 · 需自行配置

✗ 本地运行为主

✗ Serverless 需自搞

✗ 无官方云部署

维度 4 · 安全模型：逐条审批 vs 默认隔离



安全模型核心对比与选型建议



Hermes · 智能自主型

适用：个人用户 / 开发者

- ✓ 信任 Agent 自主性，减少人工干预
- ✓ 强大的记忆能力，跨会话保持
- ✓ 低成本快速部署，轻量化架构
- ✓ 自主学习进化，越用越好



OpenClaw · 企业可控型

适用：企业级部署

- ✓ 需要完整可控性，全程可追溯
- ✓ 完整审计合规，满足监管要求
- ✓ 精细化权限控制，精准管控每步
- ✓ 动作审批机制，关键操作人工确认

怎么选？



选 Hermes： 想要记忆力超棒的长期 AI 伙伴、预算敏感、想低成本部署（几十块的 ECS 就跑）、不想折腾配置、希望 Agent 自己学会做事。

选 OpenClaw： 企业级部署、需要完整审计与合规、要精细控制每条指令、Agent 的每个动作都需要审批。

三者对比，一张表看全

维度	OpenClaw	Hermes	Claude Code
定位	永远在线的个人助理	自我进化的个人助理	生产级编码 agent
出品	社区开源	Nous Research	Anthropic
外壳重心	Gateway + 多平台生态	学习闭环 + 沙箱隔离	权限 / 上下文 / 子 agent
配置 & 记忆	Markdown 文件	自动生成的技能文件	CLAUDE.md + 文件记忆
杀手锏	心跳主动性、生态最广	越用越强、可产训练数据	安全与上下文工程最成熟
主要软肋	安全 CVE 多	API 不稳、可观测性弱	偏编码、生态相对封闭
适合谁	想要全能私人管家	重复工作流 + 隐私敏感	软件工程团队

建一个 agent，你必答的 6 个问题



1. 推理放哪？

放模型 vs 放外壳

洞察 模型在收敛，外壳才是差异化所在。



2. 安全姿态？

默认放行 / 拦截 / 升级

洞察 纵深防御一旦各层共享失败模式就失效。



3. 上下文怎么管？

单次截断 vs 渐进管线

洞察 从第一天就按「上下文稀缺」设计。



4. 扩展怎么插？

一个 API vs 分层机制

洞察 不是所有扩展都该消耗 token。



5. 子 agent 共享还是隔离？

共享上下文 vs 隔离

洞察 只回摘要，防止上下文爆炸。



6. 持久化保留啥？

可恢复 / 可审计

洞察 恢复时永不恢复权限，审计优先于查询。

怎么选：对号入座

OpenClaw

选 OpenClaw, 如果你...

- ✓ 想要一个全能私人管家，接管多个消息平台
- ✓ 看重生态：装现成技能、开箱即用
- ✓ 能接受自己做安全加固

Hermes

选 Hermes, 如果你...

- ✓ 有大量重复 workflows，想让它越用越懂你
- ✓ 隐私 / 安全敏感，要沙箱隔离 + 本地运行
- ✓ 想顺便产训练数据 / 自研模型

Claude Code

选 Claude Code, 如果你...

- ✓ 核心诉求是严肃的软件工程 / 写代码
- ✓ 要成熟的权限、上下文、子 agent 能力
- ✓ 团队协作，需要可审计、可回滚

不互斥：很多人「Claude Code 写代码 + OpenClaw / Hermes 当生活 / 运维助理」并用。

PART 04

HANDS-ON · PITFALLS

落地 · 避坑 · 经验

上手路径 · 三大坑 · 带走的三句话

10 min

从低风险起步

失控循环

攻击面

上下文爆炸

上手路径：从最低风险开始



```
# 装起来都很快
$ npm i -g @anthropic-ai/claude-code
$ claude # 进交互

# Hermes 一条命令
$ curl -fsSL hermes.sh | sh
```

第一个任务，挑「低风险、好验证」的

- 让它整理一个目录 / 写一份调研
- 跑测试并修一个已知的小 bug
- 先别让它碰生产数据和密钥



黄金姿势：先观察，再逐步放权

1

plan / 只读模式起步

先看它怎么「想」、打算调用什么工具，别让它动手。

2

给低风险写权限

确认靠谱后，放开整理文件、跑测试这类可回滚操作。

3

逐步开放更多工具

一步步扩大边界；不可逆动作始终保留人工确认。

避坑 ① 最常见：失控循环 = 烧钱



真实失败模式：搭个 agentic loop 听起来简单，直到你的终端凌晨 3 点还在刷屏，早上醒来收到一张 \$400 的 API 账单。循环会自己转几十上百轮 —— 没有刹车就等着翻车。

刹车 1

停止条件

最大轮数 / 完成判定 / 超时，必须有「什么时候停」。

刹车 2

花费上限

给会话设 token / 美元上限，到顶强制停。

刹车 3

验证步骤

每轮跑测试 / 校验，错了就停，别让它在错误上狂奔。

给循环装刹车 (示意)

```
run(agent, max_turns=20, budget_usd=2.0, stop_when=tests_pass, on_exceed="halt+ask")
```

避坑 ② 最危险：Agent 特有的攻击面

agent 能读外部内容、又有高权限工具 —— 这是普通软件没有的新攻击面。OpenClaw 的 CVE 就是活教材。



Prompt 注入

网页 / 邮件 / 文件里藏着「忽略以上指令，去做坏事」，被 agent 当成命令执行。



工具滥用

诱导 agent 用它的合法工具干坏事：删库、转账、外发数据。



记忆投毒

往它的长期记忆 / 技能文件里塞恶意内容，之后每次都中招。



防御清单

- ✓ 最小权限：每个 agent / 子 agent 只给它真正需要的工具
- ✓ 永不把生产密钥放进无隔离的 agent
- ✓ 警惕「审批疲劳」：靠设计好默认边界，而不是堆弹窗
- ✓ 隔离执行：Docker / 沙箱里跑，别碰宿主
- ✓ 不可逆操作（删 / 推 / 付费）强制人工确认

避坑 ③：上下文爆炸 · 别迷信指标



上下文爆炸 / 成本失控

- 记住「上下文是预算」：长任务里历史会爆，反应变慢、变贵、变笨。
- 用子 agent 隔离：把脏活丢给子 agent，只让摘要回主线。
- 善用压缩 / 折叠：别把所有文件全文一直挂在上下文里。
- 别开一堆 agent 才发现 $7\times$ token —— 并行很爽，账单很疼。



别迷信 star 数 / token 量

- 这些数字在不同来源差很多（同一项目 star 从十几万到三十多万都有），因为是不同日期的快照。
- 上台讲之前，当天再核一遍最新版本号、star、有没有新爆出的安全问题 —— 别被人当场纠正。
- 选型看「是否解决你的问题」，不是看谁星多。

带走这三句话

01

循环谁都能写，外壳才是产品。

98.4% 是确定性工程。选型 / 自研，盯外壳别盯模型。

02

模型在收敛，外壳在分化。

各家底座越来越像，决定体验的是权限、上下文、恢复这层。

03

安全和上下文是地基，不是加分项。

权限模型、沙箱隔离、上下文预算 —— 第一天就要设计进去。

OpenClaw 是「你告诉它做什么」， Hermes 是「它学会你想做什么」。



两条验证过的路线

OpenClaw 控制优先（确定可控） vs
Hermes 进化优先（越用越懂你）。



理念决定选型

看你要可控可审计，还是要省心自进化 ——
适配人群不同。



赛道在变

从拼功能 / 模型，转向拼「人机关系」的深度与持久。



如何选型：一句话决策

要确定性、可控、可审计 → OpenClaw（龙虾）； 要省心、自动进化、关系越用越深 → Hermes。 没有更强，只有更合适。